

+ 26

The Real Story of C++26

Beyond the Headline Features

SANDOR DARGO



20
26



```
const int& f() {  
    return 42;  
}
```

Compiles today — with a warning, if you're lucky.

In C++26: **it doesn't compile.**

No reflection. No contracts. Just a bug that stops existing.

That is the real story of C++26.

Who Am I?

Sandor Dargo

- Senior Engineer at **Spotify**
- Enthusiastic blogger: sandordargo.com
- Curious oenophile
- Fortunate father of two



What you've heard about C++26

- **Reflection** — `std::meta`, splicing, compile-time introspection
- **Contracts** — preconditions, postconditions, assertions
- **Senders/Receivers** — structured async execution

Real. Transformative. Covered in a dozen other talks this week.

The big features change what's possible. The small ones change what's practical.

Five themes

1. **Compile-time expansion** — `constexpr` keeps growing
2. **Clearer correctness** — redefining "wrong code"
3. **Safety via library** — hardening the standard library
4. **Everyday expressiveness** — fixing long-standing pain points
5. **Freestanding** — C++ in more environments

I. Compile-time Expansion

- ▶ Compile-time
 - Correctness
 - Hardening
 - Expressiveness
 - Freestanding

The `constexpr` trajectory

Standard	What became <code>constexpr</code>
C++11	Simple functions, literal types
C++14	Loops, local variables
C++17	<code>if constexpr</code> , lambdas
C++20	Allocations, <code>std::string</code> , <code>std::vector</code>
C++23	More containers, <code>static_assert</code> improvements
C++26	Exceptions, placement new, virtual inheritance, structured bindings, more library

Before C++26: the dual-API problem

```
// Runtime version: real error handling
int factorial(int n) {
    if (n < 0) { throw std::invalid_argument("don't be negative"); }
    // ...
}

// Compile-time version: ad-hoc error signaling
constexpr int safe_factorial(int n) {
    if (n < 0) {
        // What do you do here? Can't throw.
        // Return -1? Static_assert? Compiler diagnostic?
    }
    // ...
}
```

Two functions. Same logic. Different error handling.

C++26: constexpr throw

```
1 // https://godbolt.org/z/oY71Gcrvo
2 constexpr int factorial(int n) {
3     if (n < 0) { throw std::invalid_argument("don't be negative"); }
4     int result = 1;
5     for (int i = 2; i <= n; ++i) {
6         result *= i;
7     }
8     return result;
9 }
10
11 int x = factorial(n); // runtime
12 static_assert(factorial(5) == 120); // compile-time too
```

- At **runtime**: **throw** → normal exception
- At **compile-time**: **throw** → compilation error with the message

```
error: uncaught exception of type
      'std::invalid_argument'; 'what()': 'don't be negative'
```

Compiler support: GCC 16

What else became `constexpr` in C++26?

Language:

- **Placement new** — flexible initialization at compile time
- `void*` **casting** — enables `std::format`, `std::function`, `std::any`
- **Structured bindings** — can now be declared `constexpr`

Library:

- `<cmath>` **and** `<complex>` — floating-point math at compile time
- `std::stable_sort`, `std::stable_partition`, `std::inplace_merge`
- **All standard containers** — not just `vector` and `string` anymore
- `std::constant_wrapper` — pass compile-time constants as regular function arguments

Recap on compile-time expansion

- **Fewer dual APIs** — one function for both contexts
- **Simpler metaprogramming** — use normal patterns, not TMP tricks
- **Better diagnostics** — `throw` messages show up as compiler errors
- **Convergence** — the mental model is just "write C++"

II. Redefining Correctness

Compile-time

► **Correctness**

Hardening

Expressiveness

Freestanding

The problem with undefined behavior

In C++ today, many programming errors result in **undefined behavior**:

- Compiler can assume it doesn't happen
- Anything can happen — nasal demons™
- No obligation to diagnose

No middle ground between "correct" and "all bets are off." If that's the deal, what does **safe** even mean?

What does "safe" even mean?

"A safe operation is one that cannot cause undefined behaviour."

— Dave Abrahams & Sean Parent

By that definition, it's surprisingly easy to write unsafe C++ — because so many common mistakes are just UB.

The simplest fix: make it ill-formed

Some UB in C++26 just becomes a **compile error**:

```
// P2748R5: returned reference bound to temporary  
const int& f() {  
    return 42; // C++23: UB – C++26: ill-formed  
}
```

```
// P3144R2: deleting incomplete types  
struct Widget; // forward declaration only  
void cleanup(Widget* p) {  
    delete p; // C++23: UB – C++26: ill-formed  
}
```

The compiler just **rejects** code that was always wrong.

Compiler support: GCC 15 · Clang 19 (returned reference already enforced in GCC 14)

Introducing: erroneous behavior

C++26 adds a **new category** between defined behavior and UB:

```
int x;           // indeterminate value
int y = x;       // C++23: undefined behavior
                 // C++26: erroneous behavior
```

Category	Compiler may eliminate?	Must diagnose?	Outcome constrained?
Defined behavior	—	—	Fully specified
Erroneous behavior	No	Encouraged	Yes
Undefined behavior	Yes	No	No

The program is wrong, but the damage is **bounded**.

Compiler support: GCC 16

Why not just "define" it?

"Just zero-initialize everything!"

- **Performance**: mandatory initialization costs cycles in hot loops
- **Bug hiding**: zero might be a valid value — the bug becomes silent
- **Erroneous behavior** is the middle ground:
 - No performance mandate
 - Diagnosis encouraged
 - Wrong code stays visibly wrong

Indeterminate values in C++26

Clearer rules for what happens with uninitialized data:

```
1 unsigned char buf[16];    // indeterminate
2 memcpy(dst, buf, 16);    // C++26: erroneous, not UB
3
4 struct S { int a; int b; };
5 S s;
6 s.a = 42;
7 int x = s.b;             // erroneous behavior
```

But what about **intentionally** uninitialized variables?

```
int x [[indeterminate]]; // opt-in: reading x before init is still UB
```

Explicit, auditable, and **your responsibility**.

Recap on redefining correctness

- **"Wrong" is no longer all-or-nothing** — there's a middle ground between correct and UB
- **Implementations can help** — without mandating overhead
- **Diagnostics improve** — wrong code is diagnosable, not just "anything goes"
- **Foundation for more** — erroneous behavior can expand to other cases

III. Library Hardening

Compile-time

Correctness

► **Hardening**

Expressiveness

Freestanding

The standard library trust model (until now)

```
std::vector<int> v = {1, 2, 3};  
v[5]; // Your problem. UB. Good luck.
```

The standard library has historically **assumed correct usage**.

Violate the preconditions? That's on you.

C++26's answer: library hardening

The library **checks its own preconditions** at runtime.

A violation **stops the program** — instead of corrupting it.

What gets hardened?

Deliberately focused on **memory safety preconditions only** — checking everything is an explicit non-goal:

```
v[5];           // bounds check
v.front();      // empty check
*opt;           // has_value check
span.first(n);  // count in range
expected.error(); // !has_value check
mdspan[i, j, k]; // all indices in extents
```

Covered: vector, array, deque, list, forward_list, string, span, string_view, optional, expected, mdspan, bitset, valarray

"A small set of checks, widely deployed, delivers far more value than an extensive set of checks that few projects enable."

— *P3471R4, Standard library hardening*

Hardening isn't new — now it's standard

Vendors have shipped hardening modes for years. P3471 R4 **standardizes** the practice.

In the standard: **a single switch** — an implementation is *hardened* or it isn't.

In practice, vendors offer **layers** (libc++ example):

Level	What it checks	Overhead
None	Nothing	Zero
Fast	Bounds, null, basic preconditions	Minimal (production)
Extensive	+ iterators, algorithmic preconditions	Moderate
Debug	Everything, including heuristics	Significant

Enable via compiler flag: `-fhardened` or `-D_LIBCPP_HARDENING_MODE=...`

Key constraint: you cannot `ignore` a hardened precondition violation.

The proof: Google's production data

Google enabled hardened libc++ across **hundreds of millions of lines of C++**:

Metric	Result
Bugs found	> 1,000 (including security-critical)
Projected bugs prevented/year	1,000–2,000
Segfault rate reduction	30% across production
Performance overhead	0.30% average

Source: Google Security Blog, Nov 2024 — "Retrofitting spatial safety to hundreds of millions of lines of C++"

0.30% — how?

- **Redundant checks are optimized away** — the compiler can prove bounds are safe
- **Each check is $O(1)$** — a comparison and a branch, nothing more
- **Profile-guided optimization** eliminates checks on hot paths
- Google even tuned a single check to drop overhead from 0.35% → 0.30%

The correctness ↔ hardening connection

Three approaches to the same goal — **shrinking the UB surface**:

1. **Ill-formed** — the compiler rejects code that was always wrong (P2748, P3144)
2. **Erroneous behavior** — the language reclassifies some UB as bounded
3. **Library hardening** — the stdlib catches misuse at runtime via contracts (P2900)

Different mechanisms, same direction: wrong code should be **diagnosable**, not a license for the compiler to do anything.

Library hardening: the takeaway

- **Earlier failure** — bugs caught at the point of misuse, not three stack frames later
- **Production-viable** — 0.30% overhead is real, proven at Google scale
- **Portable** — one standard framework, not vendor-specific hacks
- **Foundation** — first use of contracts in the stdlib, more will follow

"Library hardening is one of those features that makes you wonder why it wasn't standardized sooner."

— sandordargo.com

IV. Everyday Expressiveness

Compile-time

Correctness

Hardening

► Expressiveness

Freestanding

Function wrappers: the landscape before C++26

```
// Raw function pointer – no state  
void (*callback)(int);  
  
// std::function – allocates, copies, owns  
std::function<void(int)> f;
```

C++23 added `std::move_only_function` — owning, move-only, `const`-correct.

But what about: **non-owning, no allocation, type-erased**?

Or: **owning, copyable** — and `const`-correct?

Gaps in the taxonomy.

std::function_ref

Non-owning, non-allocating callable reference:

```
void apply(std::function_ref<int(int)> f, int x) {  
    return f(x);  
}  
  
auto lambda = [](int x) { return x * 2; };  
apply(lambda, 21); // No allocation. No copy.
```

- Like a reference to any callable
- Zero overhead beyond the indirect call
- Perfect for **callbacks you don't need to store**

Compiler support: GCC 16 · libc++ 24 (upcoming)

std::copyable_function

Owning, type-erased, guaranteed copyable:

```
std::copyable_function<int(int)> make_adder(int n) {  
    return [n](int x) { return x + n; };  
}  
  
auto f = make_adder(10);  
auto g = f; // Always works. Guaranteed.
```

Unlike `std::function`: **const** -correct — `operator()` respects `const`.

Compiler support: GCC 16

The `std::function` `const` bug

```
// https://godbolt.org/z/nKnajhzsh
const std::function<int()> f = [n = 0]() mutable { return ++n; };
f(); // Compiles! Mutates through a const object.
```

`std::function::operator()` is always `const`, even when the wrapped callable is not. A `const` wrapper that silently allows mutation.

```
const std::copyable_function<int()> g = [n = 0]() mutable { return ++n; };
g(); // Compile error: operator() is const, callable is not
```

`std::copyable_function` gets this right: `const`-ness is part of the signature. `copyable_function<int() const>` is callable through `const` — and rejects the mutable lambda at construction.

The callable taxonomy in C++26

Type	Owns?	Allocates?	Copyable?	const -correct?
Function pointer	No	No	Yes	Yes
<code>std::function</code>	Yes	Maybe	Yes	No
<code>std::function_ref</code>	No	No	Yes	Yes
<code>std::move_only_function</code>	Yes	Maybe	No	Yes
<code>std::copyable_function</code>	Yes	Maybe	Yes	Yes

Pick the right tool for the job.

The const bug, part two

The same problem hides in composite design — pointer members don't propagate `const` :

```
1 class Widget {
2     std::unique_ptr<Impl> _pimpl;
3 public:
4     void inspect() const {
5         _pimpl->mutate(); // compiles – const doesn't propagate through the pointer
6     }
7 };
8
9 const Widget w{/* ... */};
10 w.inspect(); // mutates the Impl behind our back
```

`const Widget` promises not to change the `unique_ptr` itself — but the `Impl` it points to is fair game.

`std::indirect` : Pimpl done right

```
1 class Widget {
2     std::indirect<Impl> _pimpl;
3 public:
4     void inspect() const {
5         _pimpl->mutate(); // does NOT compile – const propagates
6     }
7 };
8
9 const Widget w{/* ... */};
10 w.inspect(); // correctly rejected
```

- `const` propagates — a `const Widget` gives you a `const Impl`
- Deep copies on assignment — no custom copy constructor needed

For hierarchies: `std::polymorphic<Base>` — copies preserve the dynamic type. No `clone()`, no slicing.

Compiler support: GCC 16

`std::format` in C++26

The missing piece: **dynamic format strings** (P2918 + P3953):

```
1 // C++20/23: format string must be a compile-time constant
2 std::format("{} logged in", username);    // OK
3
4 // C++26: dynamic format strings
5 std::string fmt = load_from_config("greeting");
6 std::format(std::dynamic_format(fmt), username);
```

Pointer formatting (P2510) — `std::format("{:018}", ptr)` with zero-padding and case control.

`std::to_string` **fixed** (P2587) — `to_string(0.42)` now returns `"0.42"`, not `"0.420000"`.

Structured bindings: three limitations removed

- **Attributes** on individual bindings (P0609R3)
- **Conditions** — decompose and branch in one statement (P0963R3)
- **Packs** — structured bindings can introduce parameter packs (P1061)

Attributes on structured bindings (P0609R3)

```
auto [x, [[maybe_unused]] _, z] = get_triple();  
// Before: [[maybe_unused]] could only be applied to the whole declaration
```

You can now annotate **individual** bindings, not just the declaration as a whole.

C++26 also makes `_` a reusable unnamed placeholder (P2169).

Compiler support: GCC 15 · Clang 19

Structured bindings as conditions (P0963R3)

```
if (auto [value, ec] = std::to_chars(buf, end, 42)) {  
    // decompose AND test in one statement  
}  
while (auto [header, body] = receive_packet()) {  
    process(header, body);  
}
```

Decompose and branch — no temporary variable just to check a status code.

Compiler support: GCC 15 · Clang 21

Structured binding packs (P1061)

Structured bindings can now introduce **parameter packs**:

```
template <typename T>
void print_fields(const T& obj) {
    auto [... fields] = obj;
    (std::cout << ... << fields);
}

// Or capture some, pack the rest:
auto [first, ...rest] = get_tuple();
```

- Packs outside templates — a boundary being pushed
- Replaces metaprogramming hacks for aggregate decomposition
- Pairs naturally with **pack indexing** (`rest...[0]`)

Compiler support: GCC 16 · Clang 21

Pack indexing

The old way — want the Nth type in a pack?

```
template<std::size_t N, typename T, typename... Ts>
struct nth_type : nth_type<N - 1, Ts...> {};

template<typename T, typename... Ts>
struct nth_type<0, T, Ts...> {
    using type = T;
};

using third = nth_type<2, int, double, char, bool>::type;
```

C++26:

```
template<typename... Ts>
using third = Ts...[2];
```

That's it.

Compiler support: GCC 15 · Clang 19

Recap on pack indexing

- **Simpler templates** — direct access replaces recursion
- **Faster compilation** — fewer template instantiations
- **More readable** — intent is obvious
- **Lowers the bar** — variadic templates become more approachable

`=delete("reason")`

Before — the compiler says *what*, not *why*:

```
error: use of deleted function 'NonCopyable::NonCopyable(const NonCopyable&).'
```

C++26 — the author explains the intent:

```
NonCopyable(const NonCopyable&)  
    = delete("Manages unique resources; use move instead.");
```

```
error: use of deleted function 'NonCopyable::NonCopyable(const NonCopyable&)'  
      "Manages unique resources; use move instead."
```

Same pattern as `[[deprecated]]`, `[[nodiscard]]`, and `static_assert` (also C++26!).

Compiler support: GCC 15 · Clang 19

V. Freestanding

Compile-time

Correctness

Hardening

Expressiveness

► Freestanding

What is freestanding?

A subset of C++ that works **without an OS**:

- No `<iostream>`, no `<filesystem>`, no exceptions (typically)
- Embedded systems, kernels, bootloaders, firmware

Historically: very little of the standard library was available.

C++26 freestanding expansion

More of the standard library is now freestanding:

- `<optional>`, `<expected>`, `<variant>`
- `<string_view>`, `<utility>`, `<tuple>`
- `<algorithm>` (non-allocating subset), more of `<type_traits>`
- `<inplace_vector>` — fixed-capacity vector, no heap allocation

```
// embedded codebase, pre-C++26  
struct MaybeConfig { bool has; Config value; }; // hand-rolled  
  
// C++26 freestanding  
std::optional<Config> load_config(); // just C++
```

The principle: if it doesn't need an OS, it should be freestanding.

Compiler support: GCC 14 · MSVC 19.42

`std::inplace_vector` : no heap required

Fixed capacity, stack storage, `std::vector` API:

```
1 // hand-rolled
2 struct SensorLog { int buf[16]; int count; };
3 void push(SensorLog& s, int v) { s.buf[s.count++] = v; } // no bounds check
4
5 // Boost – better, but a dependency
6 boost::container::static_vector<int, 16> log;
7
8 // C++26 – standardized, constexpr, freestanding
9 std::inplace_vector<int, 16> log;
10 log.push_back(reading); // throws std::bad_alloc if full
```

- No allocator, no heap — lives entirely on the stack
- If `T` is trivially copyable, so is the whole `inplace_vector`
- `constexpr`-friendly, freestanding — all themes converge

`inplace_vector` : three ways to push

```
1 v.push_back(x);           // throws std::bad_alloc if full
2
3 auto result = v.try_push_back(std::move(s));
4 if (!result) {
5     // s is NOT moved-from – guaranteed
6 }
7
8 v.unchecked_push_back(x); // precondition: size() < capacity()
```

- `push_back` — safe default, throws on overflow
- `try_push_back` — fallible, does not consume the argument on failure
- `unchecked_push_back` — no check, maximum performance, UB if full

Compiler support: GCC 15 · MSVC 19.43

Recap on freestanding

- **Less reinventing** — use `std::optional`, `std::inplace_vector` instead of your own
- **Portability** — same code across hosted and freestanding
- **Ecosystem alignment** — libraries can depend on standard types
- **Same vocabulary** — hosted and embedded speak the same C++

Putting It Together

It's not a feature list — it's a direction

These five themes reinforce each other:

- **Compile-time + correctness** → catch errors before runtime
- **Correctness + hardening** → catch the rest at runtime
- **Hardening + expressiveness** → safer APIs you want to use
- **Expressiveness + freestanding** → modern C++ everywhere

Make correct C++ easier to write, and incorrect C++ easier to catch.

What to try first on Monday

1. **Recompile with** `-std=c++26` — dangling references and incomplete-type deletes become compile errors for free
2. **Hardening flags** — turn them on in your CI today. **No C++26 required** — works on your current codebase
3. **constexpr throw** — unify your compile-time/runtime functions
4. `std::function_ref` — replace your next callback parameter
5. **Pack indexing** — simplify your variadic templates

+ 26

The Real Story of C++26

Beyond the Headline Features

SANDOR DARGO



20
26

